Assignment 08

Problem 0: Daily Sales

Write a program that prompts the user to enter in a series of sales figures for a 7 day period. The program should store these figures in a list and then display the following data:

- Total sales for the week
- Average daily sales amount
- The best sales day
- The worst sales day

Ensure that the user enters in values that are 0 or greater. If not, you should ask them to re-enter the data for that day. Here's an example running of this program:

```
Sales for day 1: 100
Sales for day 2: 200
Sales for day 3: -10
Sorry, that is not a valid amount. Please try again.
Sales for day 3: 150
Sales for day 4: 750
Sales for day 5: 99
Sales for day 6: 101
Sales for day 7: 125

Total sales: 1525.0
Average sales per day: 217.86
Highest sales day: 750.0
Lowest sales day: 99.0
```

Submit to Brightspace using the following filename: *LastnameFirstname_assign8_part0.py*

Problem 0 Points (17):

- Uses a loop to validate input (5 points)
- For each of 7 inputs it collects, appropriate number is printed; e.x. first prompt is "Sales for day 1:", second prompt is "Sales for day 2", etc. (5 points)
- At the end of the program, total sales, average sales, highest sales, and lowest sales are printed (7 points)

Problem 1: My Contacts

For this problem, you will be creating a contact list based on names and email addresses. To do so, you will ask the user how many contacts he or she wants to include, and use two lists to keep track of the contacts' names and associated email addresses. The user must enter at least one contact.

The contact name must have both a first name and a last name (i.e. a sequence of characters, followed by a space, followed by another sequence of characters) and only include alphabetical characters.

The email addresses must be properly formatted – i.e. anemail@website.com

Make sure the following conditions are met for the email (the first part of an email is called a "username"):

- The username starts with an alphabetical character
- The username may only include letters and digits
- The email must contain an @ sign after the username
- The email must contain a domain name between the @ sign and the .com
- The domain can only have letters
- The email must end in .com

Once you have collected all of the contacts, you must print out the names and email addresses of each contact (hint: you will need two lists to do this properly).

Here is a sample running of the program:

```
How many contacts would you like to add? -3
Invalid, please try again
How many contacts would you like to add? 3
Please enter the name of contact 1: George
Invalid, please try again
Please enter the name of contact 1: George Washington
Please enter the email of contact 1: gwash@aol.com
Please enter the name of contact 2: Thomas Jefferson
Please enter the email of contact 2: tommyj@gmail
Invalid, please try again
Please enter the email of contact 2: tommyj@gmail.com
Please enter the name of contact 3: Benjamin123
Invalid, please try again
Please enter the name of contact 3: Benjamin Franklin
```

```
Please enter the email of contact 3: bennyboy123@gmail.com
Thanks! Here is your address book:
Name: George Washington, Email: g.wash@aol.com
Name: Thomas Jefferson, Email: tommyj@gmail.com
Name: Benjamin Franklin, Email: bennyboy123@gmail.com
```

Submit to Brightspace using the following filename: *LastnameFirstname_assign8_part1.py*

Problem 1 Points (35):

- Loop is used to only allow a positive number of contacts (2 points)
- At the end of the program, names and corresponding emails are printed out (3 points)
- For each of input it collects, appropriate number is printed; e.x. first prompt is "Name/email for day 1:", second prompt is "Name/email for day 2", etc. (3 points)
- Two lists are used to properly store user input (3 points)
- Use the following tests to validate the name rules (3 points each):
    - Alexander Hamilton – valid
    - Alexander – invalid
    - Alexander123 – invalid
    - Alexander Hamilton123 – invalid
- Use the following tests to validate the email rules (3 points each):
    - 123alexh@gmail.com – invalid
    - Alexh.com – invalid
    - alexh@msn.com – valid
    - alexh@gmail.net – invalid

Problem 2a: Fast Food Restaurant

Note that the programs you will be writing for Part2a through Part 2f build on one another. You will want to attempt these parts in order. You can also feel free to save all of your work in one big file (call it "LastNameFirstName_assign8_part2.py")

Imagine that you are helping to build a store management system for a fast food restaurant. Given the following lists, write a program that asks the user for a product name. Next, find out if the restaurant sells that item and report the status to the user. Allow the user to continually inquire about product names until they elect to quit the program.

Here are some lists to get you started:

```
# product lists
product_names = ["soft drink", "onion rings", "small fries"]
product_costs = [0.99, 1.29, 1.49]
```

And here's a sample running of your program:

```
(s)earch for product or (q)uit: s
Enter a product name: soft drink
We sell soft drink at 0.99 per unit

(s)earch for product or (q)uit: s
Enter a product name: salad
Sorry, we don't sell salad

(s)earch for product or (q)uit: voldemort
Invalid option, try again

(s)earch for product or (q)uit: q
See you soon!
```

Problem 2b

Next, extend your program so that it keeps track of how many products the store currently has to sell. Default the store to have 10 soft drinks, 5 onion rings and 20 small fries. Hint: you might want to create a new list to store this data! - once you have this information in place you should modify your program to do the following:

- Update the search feature to include a report of how many products the store currently has
- Add a new feature that lists ALL products, their prices and the amount the store has available to sell

Here's a sample running of your program:

```
(s)earch, (l)ist or (q)uit: s
Enter a product name: soft drink
We sell soft drink at 0.99 per unit
We currently have 10 in stock

(s)earch, (l)ist or (q)uit: l
Product         Price  Quantity
soft drink       0.99   10
onion rings      1.29   5
```

```
small fries      1.49   20

(s)earch, (l)ist or (q)uit: s
Enter a product name: pikachu
Sorry, we don't sell pikachu

(s)earch, (l)ist or (q)uit: q
See you soon!
```

Problem 2c

Next, add in an "add" feature that lets users add a new product to the store. When you add a product you will need to ask the user for the name of the product, the cost of the product and how many items the store has available to sell. Validate this data - you cannot add a product that already exists and both the price and inventory amount must be positive. Here's a sample running of the program:

```
(s)earch, (l)ist, (a)dd or (q)uit: l
Product                   Price  Quantity
soft drink                 0.99        10
onion rings                1.29         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd or (q)uit: a
Enter a new product name: onion rings
Sorry, we already sell that product. Try again.
Enter a new product name: dbl onion rings
Enter a product cost: 0
Invalid cost. Try again.
Enter a product cost: 1.49
How many of these products do we have? -5
Invalid quantity. Try again.
How many of these products do we have? 100
Product added!

(s)earch, (l)ist, (a)dd or (q)uit: l
```

```
Product                   Price  Quantity
soft drink                 0.99        10
onion rings                1.29         5
small fries                1.49        20
dbl onion rings            1.49   100

(s)earch, (l)ist, (a)dd or (q)uit: s
Enter a product name: dbl onion rings
We sell dbl onion rings at 1.49 per unit
We currently have 100 in stock

(s)earch, (l)ist, (a)dd or (q)uit: q
See you soon!
```

Problem 2d

Next, add in a "remove" feature that lets users remove products from the database. Here's a sample running of the program:

```
(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: l
Product                   Price  Quantity
soft drink                 0.99        10
onion rings                1.29         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: r
Enter a product name: pikachu
Product doesn't exist. Can't remove.

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: r
Enter a product name: small fries
Product removed!

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: l
Product                   Price  Quantity
```

```
soft drink                 0.99        10
onion rings                1.29         5

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: r
Enter a product name: soft drink
Product removed!

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: l
Product                   Price  Quantity
onion rings                1.29         5

(s)earch, (l)ist, (a)dd, (r)emove or (q)uit: q
See you soon!
```

Problem 2e

Add in the ability to modify a product. Users should be able to modify the name, cost or quantity of a product. Note that you will need to perform data validation as needed. Here's a sample running of your program:

```
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: l
Product                   Price  Quantity
soft drink                 0.99        10
onion rings                1.29         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: u
Enter a product name: burger
Product doesn't exist. Can't update.

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: u
Enter a product name: soft drink
What would you like to update?
(n)ame, (c)ost or (q)uantity: n
Enter a new name: onion rings
Duplicate name!
```

```
Enter a new name: soft drink royale
Product name has been updated

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: l
Product                   Price  Quantity
soft drink royale          0.99        10
onion rings                1.29         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: u
Enter a product name: onion rings
What would you like to update?
(n)ame, (c)ost or (q)uantity: c
Enter a new cost: 0
Invalid cost!
Enter a new cost: 9.50
Product cost has been updated

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: l
Product                   Price  Quantity
soft drink royale          0.99        10
onion rings                9.50         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: u
Enter a product name: small fries
What would you like to update?
(n)ame, (c)ost or (q)uantity: q
Enter a new quantity: -500
Invalid quantity!
Enter a new quantity: 30
Product quantity has been updated

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: l
```

```
Product                   Price  Quantity
soft drink royale          0.99        10
onion rings                9.50         5
small fries                1.49        30

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: u
Enter a product name: soft drink royale
What would you like to update?
(n)ame, (c)ost or (q)uantity: nothing
Invalid option

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate or (q)uit: q
See you soon!
```

Problem 2f

Finally, add in a reporting feature to your program that finds the highest priced product, the lowest priced product and the total value of all inventory in the store (product cost * product quantity). Here's a sample running of your program:

```
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: l
Product                   Price  Quantity
soft drink                 0.99        10
onion rings                1.29         5
small fries                1.49        20

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: e
Most expensive product:      1.49 (small fries)
Least expensive product:     0.99 (soft drink)
Total value of all products: 46.15

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: q
See you soon!
```

Problem 2 Points (48):
2 points each (in some of these, multiple cases will be tested, adding up to 24 tests/48 points):

- Invalid options not allowed

- Searching for a product that does not exist results in “Sorry, we don’t sell [product name]”
- Searching for a product that does exist results in printing the product name, price, and quantity
- Products can be added with proper cost and quantity (i.e. positive values)
- Products that are already in the inventory cannot be added
- Invalid costs are not accepted when adding products
- Invalid quantities are not accepted when adding products
- Products that aren’t in inventory can’t be removed
- Products that are in inventory can be removed
- Products that aren’t in inventory can’t be updated
- Products that are in inventory can have cost updated
- Products that are in inventory can have quantity updated
- When products are reported, the most and least expensive items/costs are printed, and the sum of all products is printed
- ‘q’ input causes the program to end